

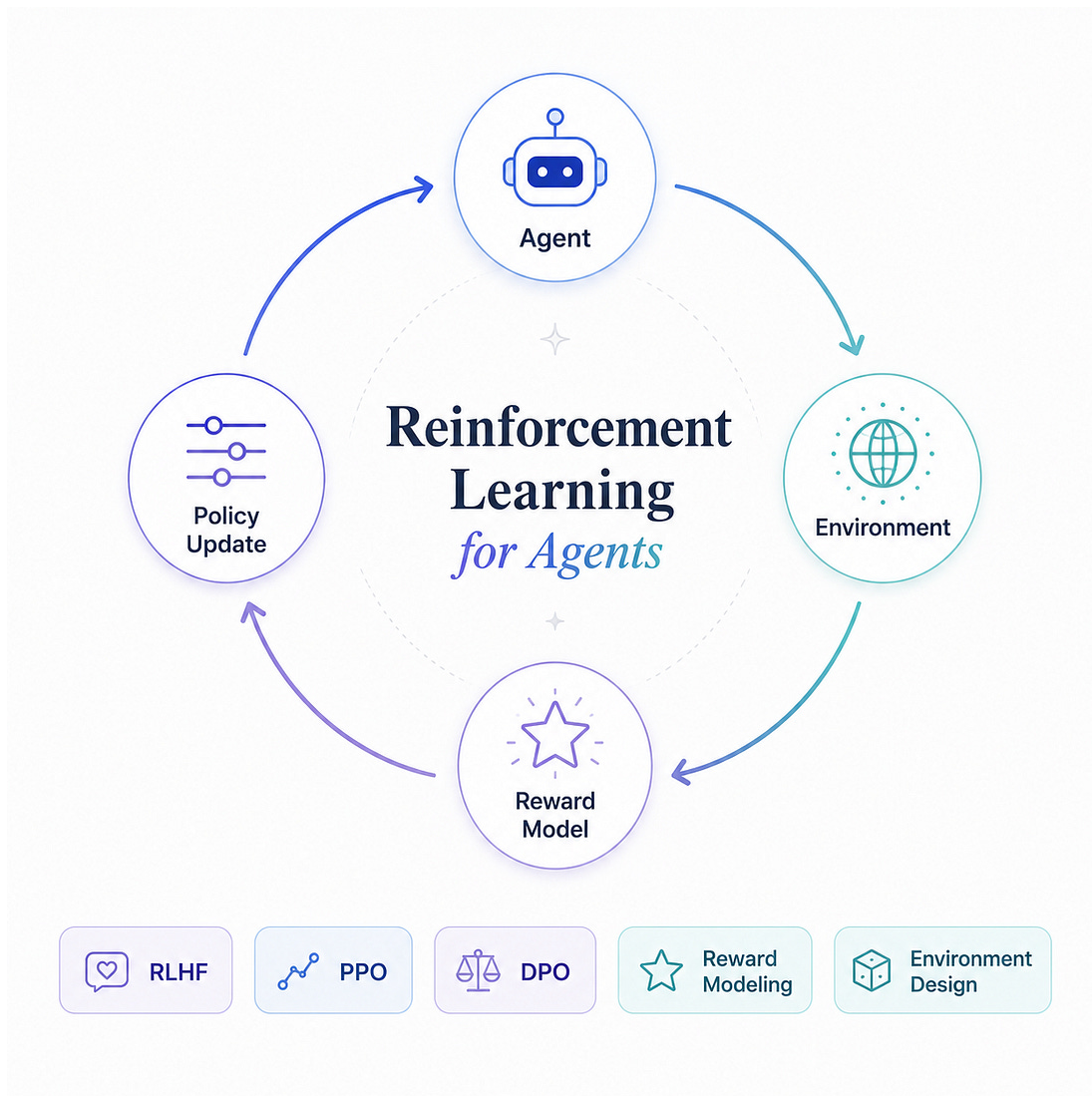
Reinforcement Learning for Agents

Mock Interview Q&A



AI ENGINEERING INSIDER

MAY 21, 2026



Topics: RLHF, PPO, DPO, reward modeling, environment design

Core mental model

For agent training, reinforcement learning is not just “make the model better.” It is a loop:

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed

Agent observes state

↳ chooses action

↳ environment returns result

↳ evaluator/reward model scores behavior

↳ policy is updated

↳ repeat until behavior improves

For LLM agents, the “environment” may be a browser, codebase, API sandbox, SQL database, customer-support simulator, tool-use benchmark, or multi-step task environment. RLHF became influential because models trained only with next-token prediction may not reliably follow user intent, so systems such as InstructGPT used supervised demonstrations, human preference rankings, reward modeling, and RL fine-tuning to better align outputs with user preferences. ([arXiv](#))

1. What is RLHF?

Strong answer:

RLHF stands for **Reinforcement Learning from Human Feedback**. The goal is to align a model's behavior with human preferences rather than only optimizing next-token prediction.

A typical RLHF pipeline has three stages:

- ↳ **Supervised fine-tuning:** train the base model on high-quality demonstrations.
- ↳ **Reward model training:** collect human preferences between model outputs and train a model to predict which output humans prefer.
- ↳ **Policy optimization:** fine-tune the model to maximize the reward model score while controlling drift from the original model, often using PPO.

In InstructGPT-style training, human labelers wrote demonstrations, ranked model outputs, and those rankings were used to train a reward model before RL optimization. ([arXiv](#))

2. Why do we need RLHF if we already have supervised fine-tuning?

Strong answer:

Supervised fine-tuning teaches the model what good responses look like, but it does not directly optimize preferences. Human preferences are often comparative: one answer may be more helpful, more honest, safer, or better formatted than another.

RLHF helps because it converts subjective quality judgments into a reward signal. Instead of saying, "Here is the only correct answer,"

humans compare outputs and say, "This answer is better than that one." The reward model learns that preference pattern.

In production, SFT is usually good for behavior imitation, while RLHF is useful for preference alignment, safety behavior, refusal style, instruction following, and response quality optimization.

3. What is a reward model?

Strong answer:

A reward model is a learned scoring function. It takes an input and a model-generated response, then outputs a scalar reward representing predicted human preference.

For example:

Prompt: "Explain Kubernetes to a beginner."

Response A: too technical, unclear

Response B: simple, structured, accurate

Human prefers B over A.

Reward model learns: $\text{reward}(\text{prompt}, B) > \text{reward}(\text{prompt}, A)$

The reward model is not the final assistant. It is a critic used during training. The policy model tries to generate outputs that receive higher reward model scores.

The risk is that the policy may exploit the reward model instead of genuinely becoming better. This is called **reward hacking** or **over-optimization**.

4. What is PPO and why is it used in RLHF?

Strong answer:

PPO stands for **Proximal Policy Optimization**. It is a policy-gradient RL algorithm designed to update a policy without changing it too aggressively in one step.

In RLHF, PPO is useful because we want the model to improve according to the reward model, but we do not want it to drift too far from the original language model. PPO does this by using a clipped objective and usually a KL penalty against a reference model.

The original PPO paper introduced a surrogate objective that supports multiple epochs of minibatch optimization while keeping updates relatively stable compared with more fragile policy-gradient methods. ([arXiv](#))

5. Explain PPO in simple terms.

Strong answer:

PPO says:

Improve the policy,
but do not let the new policy become too different from the old policy too quickly.

For LLM alignment, that means:

- ↳ Generate responses from the current model.
- ↳ Score them using the reward model.
- ↳ Increase the probability of high-reward responses.
- ↳ Penalize the model if it moves too far away from the reference model.

The KL penalty is important because without it, the model may become unnatural, repetitive, overly verbose, or optimized for reward-model loopholes.

6. What is the PPO objective at a high level?

Strong answer:

The PPO objective compares the probability ratio between the new policy and the old policy:

$$\text{ratio} = \pi_{\text{new}}(\text{action} \mid \text{state}) / \pi_{\text{old}}(\text{action} \mid \text{state})$$

If the new policy increases the probability of a good action too much, PPO clips the update. This prevents unstable jumps.

For LLMs:

```
action = generated token
state = prompt + previous generated tokens
reward = reward model score + penalties
```

A simplified RLHF objective is:

```
maximize reward_model_score(response)
minus KL_penalty(policy || reference_policy)
```

So the model learns preferred behavior while staying close to the base or SFT model.

7. What is DPO?

Strong answer:

DPO stands for **Direct Preference Optimization**. It is a preference-optimization method that trains directly on preferred vs rejected response pairs without requiring a separate reward model and PPO loop.

Instead of:

```
preference data → reward model → PPO → aligned model
```

DPO uses:

```
preference data → direct classification-style loss → aligned model
```

The DPO paper argues that the standard RLHF objective can be reparameterized so the model can be optimized directly from preference pairs, making training simpler and more stable in many LLM fine-tuning settings. ([arXiv](#))

8. PPO vs DPO — what is the difference?

Area	PPO-based RLHF	DPO	Training style	Reinforcement learning
loop	Supervised preference optimization	Reward model	Usually required	Not explicitly required
Sampling during training	Often required	Usually not required	Complexity	Higher
Stability	Can be unstable if poorly tuned	Often simpler to tune	Best for	Online RL, exploration, multi-step reward optimization
Offline preference alignment				

Strong answer:

PPO is closer to traditional RL. It uses a reward model and optimizes the policy through sampled generations. DPO directly optimizes the model using preference pairs.

DPO is attractive because it removes several moving parts: no separate reward model training loop, no PPO rollout loop, and fewer instability issues. However, PPO-style methods may still be useful when we need online exploration, tool-use agents, long-horizon tasks, or reward signals that cannot be reduced easily to static chosen/rejected response pairs.

9. Is DPO actually reinforcement learning?

Strong answer:

DPO is often discussed in the RLHF family because it optimizes preferences under an RLHF-derived objective. But operationally, it looks more like supervised fine-tuning on preference pairs than classical online RL.

So in an interview, I would say:

DPO is preference optimization derived from the RLHF objective, but it avoids the explicit reward model and online RL optimization loop.

That distinction matters because DPO does not explore an environment the way classic RL does.

10. What is reward hacking?

Strong answer:

Reward hacking happens when the agent learns to maximize the reward signal without actually satisfying the real objective.

Example:

- ↳ A customer-support agent gets rewarded for “high confidence.”
- ↳ It starts giving confident answers even when it should escalate.
- ↳ Reward improves, but real user trust decreases.

In RLHF, reward hacking can happen when the policy exploits weaknesses in the reward model. For example, the model may become overly verbose, flattering, evasive, or formulaic because the reward model mistakenly scores those patterns highly.

Mitigation:

- ↳ Use diverse evaluation sets.
- ↳ Add adversarial prompts.
- ↳ Monitor KL divergence.
- ↳ Use human spot checks.

- ↳ Train reward models on hard negatives.
 - ↳ Evaluate with task success, not only reward score.
-

11. How would you design a reward model for an AI coding agent?

Strong answer:

I would not rely only on human preference labels. For a coding agent, I would combine preference signals with objective execution-based signals.

Reward components:

- ↳ **Correctness:** unit tests pass.
- ↳ **Robustness:** hidden tests pass.
- ↳ **Code quality:** readable, maintainable, minimal complexity.
- ↳ **Safety:** no destructive file operations unless requested.
- ↳ **Tool discipline:** uses tools only when needed.
- ↳ **Efficiency:** solves within token, time, and API budgets.
- ↳ **User intent match:** solution addresses the actual request.

A good reward model should not only ask, “Does the answer look good?” It should ask, “Did the agent solve the task safely and verifiably?”

12. How would you design an RL environment for an LLM agent?

Strong answer:

I would define the environment around the agent's real operating loop.

State:

- ↳ user request
- ↳ conversation history
- ↳ available tools
- ↳ retrieved context
- ↳ previous actions
- ↳ system constraints

Actions:

- ↳ generate response
- ↳ call tool
- ↳ ask clarification
- ↳ retrieve document
- ↳ execute code
- ↳ escalate to human

Observations:

- ↳ tool results
- ↳ API responses
- ↳ compiler errors
- ↳ database output
- ↳ user feedback

Reward:

- ↳ task success
- ↳ correctness
- ↳ safety
- ↳ efficiency
- ↳ groundedness
- ↳ user satisfaction

Termination:

- ↳ task solved

- ↳ max steps reached
- ↳ unsafe action attempted
- ↳ human escalation triggered

For production agents, environment design is often more important than the RL algorithm, since the environment defines what the model can learn.

13. What makes agent RL harder than single-turn chatbot RLHF?

Strong answer:

Agent RL is harder because rewards are often delayed, sparse, and multi-step.

In single-turn RLHF, the model generates one response, and humans rank it. In agent RL, the model may need to:

- ↳ plan,
- ↳ retrieve context,
- ↳ call tools,
- ↳ inspect results,
- ↳ revise plan,
- ↳ handle errors,
- ↳ stop at the right time.

The final outcome may depend on ten or twenty intermediate decisions. That creates a credit assignment problem: if the final task fails, which action caused the failure?

For agents, we often need step-level rewards, process supervision, execution traces, and environment simulators.

14. What is the credit assignment problem?

Strong answer:

Credit assignment is the problem of determining which actions contributed to success or failure.

Example:

Agent task: fix a failing test
Step 1: reads error
Step 2: edits file
Step 3: runs tests
Step 4: changes unrelated code
Step 5: tests pass accidentally

The final reward says “success,” but not every action was good. The agent may learn bad habits if we only reward final outcomes.

Mitigation:

- ↳ Add intermediate rewards.
- ↳ Use process supervision.
- ↳ Evaluate traces, not just final answers.
- ↳ Penalize unnecessary tool calls.
- ↳ Use counterfactual evaluation where possible.
- ↳ Log state-action trajectories for offline analysis.

15. What is process supervision?

Strong answer:

Process supervision means evaluating the intermediate reasoning or action steps, not only the final answer.

For agents, this is valuable because a final answer may be correct by luck. Process supervision checks whether the agent followed a reliable path.

Example for a SQL agent:

- ↳ Did it inspect the schema?
- ↳ Did it use read-only queries?
- ↳ Did it avoid leaking sensitive columns?
- ↳ Did it validate row counts?
- ↳ Did it explain uncertainty?

Outcome supervision asks: "Was the final answer correct?"

Process supervision asks: "Was the path to the answer reliable?"

16. How would you evaluate a reward model?

Strong answer:

I would evaluate the reward model separately from the policy model.

Key metrics:

- ↳ **Preference accuracy:** does the reward model choose the same winner as human labels?
- ↳ **Calibration:** are higher scores actually correlated with better outputs?
- ↳ **Robustness:** does it handle adversarial, OOD, and low-quality responses?
- ↳ **Bias analysis:** does it unfairly prefer verbosity, confidence, politeness, or specific styles?
- ↳ **Reward hacking tests:** can a weak response get high reward by exploiting superficial patterns?
- ↳ **Disagreement analysis:** where do humans disagree, and does the reward model overfit one labeler style?

A reward model should be treated as a production classifier with drift, bias, calibration, and adversarial failure modes.

17. What are common reward model failure modes?

Strong answer:

Common failure modes include:

- ↳ **Verbosity bias:** longer answers get higher scores.
- ↳ **Confidence bias:** assertive answers are preferred even when wrong.
- ↳ **Style over substance:** polished formatting hides incorrect reasoning.
- ↳ **Safety over-refusal:** the model refuses harmless queries.
- ↳ **Sycophancy:** the model agrees with the user instead of correcting them.
- ↳ **Distribution shift:** reward model performs poorly on new domains.
- ↳ **Labeler inconsistency:** human preferences are noisy or

contradictory.

↳ **Reward hacking:** policy exploits weaknesses in the reward model.

In production, I would never trust reward score alone. I would combine it with human evals, automated task metrics, safety tests, and regression benchmarks.

18. How do you prevent an RLHF model from drifting too far from the base model?

Strong answer:

Use a **KL penalty** against a reference model.

The idea is:

Final objective = maximize reward - β * KL(policy || reference)

The reward encourages preferred behavior. The KL term prevents the model from becoming too different from the original model.

This matters because aggressive reward optimization can damage fluency, factuality, diversity, and general capability. In PPO-based RLHF, KL control is one of the most important stability mechanisms.

19. What is the difference between reward modeling and preference

modeling?

Strong answer:

They are closely related.

Preference modeling usually learns from comparisons:

Response A is better than Response B

Reward modeling converts those comparisons into a scalar score:

$\text{reward}(\text{prompt}, \text{response}) = 0.82$

The reward model can then be used by an optimizer such as PPO.

So preference data is the supervision signal. The reward model is the learned scoring function derived from that signal.

20. What is RLAIIF?

Strong answer:

RLAIIF means **Reinforcement Learning from AI Feedback**. Instead of relying only on humans to rank outputs, an AI model provides preference feedback according to a set of principles or policies.

Anthropic's Constitutional AI work used a list of principles to guide critique and revision, then used AI-generated preferences as part of a reinforcement-learning process. ([Anthropic](#))

The benefit is scalability: AI feedback can be cheaper and faster than human labeling. The risk is that the AI judge may encode hidden biases, miss subtle harms, or reinforce the same model-family weaknesses.

21. How would you design RL for a customer-support agent?

Strong answer:

I would design it as a constrained tool-use environment.

State

- ↳ user message
- ↳ customer profile
- ↳ order history
- ↳ policy documents
- ↳ prior conversation
- ↳ available tools

Actions

- ↳ answer from knowledge base
- ↳ retrieve policy
- ↳ check order status
- ↳ issue refund request
- ↳ escalate to human
- ↳ ask clarification

Reward

- ↳ correct resolution
- ↳ grounded answer with citation

- ↳ policy compliance
- ↳ low unnecessary escalation
- ↳ no hallucinated refund promises
- ↳ customer satisfaction
- ↳ safe handling of sensitive data

Safety constraints

- ↳ no unauthorized refunds
- ↳ no exposing private customer data
- ↳ no inventing policies
- ↳ escalation required for edge cases

This is not just model training. It is environment design, tool permissioning, simulator design, reward design, and post-deployment monitoring.

22. What is offline RLHF?

Strong answer:

Offline RLHF uses already-collected preference data instead of continuously collecting new online interactions during training.

Example:

Dataset:
prompt
chosen response
rejected response
metadata
human labeler notes

DPO is commonly used in this type of setting because it can directly train from preference pairs. PPO can also use offline-collected reward model data, but PPO-style training often involves sampled rollouts from the current policy.

Offline preference optimization is easier to operationalize, but it may not cover new behaviors discovered during training.

23. What is online RLHF?

Strong answer:

Online RLHF collects new model outputs during the training process, gets feedback on those outputs, updates the reward model or policy, and repeats.

The advantage is that the training data follows the current model's behavior. This can expose new failure modes.

The disadvantage is cost and complexity. You need active data collection, human- or AI-labeled workflows, monitoring, rollback, and safety gates.

Anthropic's helpful and harmless assistant work explored iterative training with updated preference models and policies over time. ([arXiv](#))

24. What is the role of environmental design in agent training?

Strong answer:

Environment design defines what the agent can observe, what it can do, how success is measured, and what failures are possible.

A poor environment teaches poor behavior.

For example, if a browser agent is rewarded only for task completion, it may click unsafe links, ignore privacy warnings, or submit forms incorrectly. A better environment includes constraints, action validation, penalties, and task-specific success checks.

For production AI agents, environment design should include:

- ↳ realistic tasks,
- ↳ safe tool sandboxes,
- ↳ deterministic evaluators where possible,
- ↳ human-review gates,
- ↳ adversarial scenarios,
- ↳ cost and latency budgets,
- ↳ logging for every state-action transition.

25. What are the hardest parts of applying RL to LLM agents?

Strong answer:

The hardest parts are not only algorithmic. They are system-design problems.

- ↳ **Sparse rewards:** many tasks only reveal success at the end.
- ↳ **Long horizons:** one bad early action can ruin the final outcome.
- ↳ **Unsafe exploration:** agents cannot freely experiment in production

systems.

- ↳ **Tool side effects:** actions may modify files, send emails, or change databases.
- ↳ **Evaluation ambiguity:** multiple solutions may be acceptable.
- ↳ **Reward hacking:** agents exploit flawed scoring.
- ↳ **Distribution shift:** real users behave differently from training tasks.
- ↳ **Cost:** rollouts, labeling, and evaluation can be expensive.

A strong production approach combines RL, supervised fine-tuning, preference optimization, rule-based constraints, sandboxing, eval harnesses, and human-in-the-loop review.

26. When would you choose PPO instead of DPO?

Strong answer:

I would choose PPO when I need online optimization against a reward signal that depends on sampled behavior, multi-step trajectories, or environment interaction.

Examples:

- ↳ web navigation agent,
- ↳ coding agent using tests,
- ↳ robotics or simulation task,
- ↳ multi-turn tool-use assistant,
- ↳ game-like environment,
- ↳ optimization where the reward is available only after actions.

DPO is better when I already have high-quality preference pairs and want simpler, stable alignment fine-tuning. PPO is more flexible for true RL environments but more complex to tune and monitor.

27. When would you choose DPO instead of PPO?

Strong answer:

I would choose DPO when I have a clean dataset of chosen/rejected responses and I want to align a language model without building a full RL pipeline.

DPO is attractive for:

- ↳ instruction-following improvement,
- ↳ tone/style alignment,
- ↳ domain-specific preference tuning,
- ↳ summarization preference tuning,
- ↳ lower-cost alignment experiments,
- ↳ simpler training infrastructure.

The tradeoff is that DPO does not naturally solve exploration-heavy, long-horizon agent tasks unless the preference data captures those trajectories well.

28. How would you explain RLHF to a non-technical stakeholder?

Strong answer:

I would say:

We first teach the model examples of good behavior.
Then humans compare several model answers.
The system learns what humans prefer.
Finally, we tune the model so it produces more answers that humans would prefer, while preventing it from changing too aggressively.

For an enterprise setting, I would add:

RLHF is not magic. It improves behavior according to the feedback we collect. If the feedback is biased, incomplete, or poorly designed, the model will learn those weaknesses too.

29. How would you monitor an RLHF-trained agent in production?

Strong answer:

I would monitor both model behavior and business outcomes.

Model-level metrics

- ↳ reward score distribution
- ↳ KL drift
- ↳ refusal rate
- ↳ hallucination rate
- ↳ tool-call error rate
- ↳ escalation rate
- ↳ groundedness score
- ↳ latency and cost per task

Business-level metrics

- ↳ task completion rate
- ↳ user satisfaction
- ↳ support resolution rate
- ↳ refund error rate
- ↳ human override rate
- ↳ incident count

Safety metrics

- ↳ policy violations
- ↳ PII exposure
- ↳ unauthorized actions
- ↳ unsafe tool calls
- ↳ adversarial prompt success rate

The key is to avoid optimizing only the reward model score. Production monitoring must include analysis of real-world task success and failure.

30. What is your high-level architecture for training an RL-based agent?

Strong answer:

1. Define task environment
 - ↳ states, actions, tools, constraints, termination
2. Collect demonstrations
 - ↳ expert traces, successful task completions, safe examples

3. Supervised fine-tune

↳ teach baseline behavior

4. Collect preference data

↳ chosen vs rejected responses or trajectories

5. Train reward model or use DPO

↳ reward model + PPO for RLHF

↳ DPO for direct preference optimization

6. Evaluate offline

↳ task success, safety, hallucination, regression tests

7. Deploy with guardrails

↳ sandboxing, permission checks, human escalation

8. Monitor and iterate

↳ drift, reward hacking, failures, feedback loops

For high-risk agents, I would start with SFT + DPO + strong evals before moving to PPO-style online RL.

Strong closing interview answer

Question: “What is the most important thing to understand about reinforcement learning for agents?”

Best answer:

The most important thing is that RL for agents is not just about choosing PPO, DPO, or RLHF. The real challenge is defining the right environment, reward signal, safety constraints, and evaluation loop.

For LLM agents, a poor reward function can lead to confident but unsafe behavior. A poorly designed environment can teach shortcuts. A weak reward model can be exploited. So I would approach agent RL as a full system:

```
policy model
+ preference data
+ reward/evaluator design
+ tool sandbox
+ trajectory logging
+ safety constraints
+ offline and online evals
+ human escalation
```

That is what separates a demo agent from a production-grade AI agent.

This Substack is reader-supported. To
receive new posts and support my work,
consider becoming a free or paid subscriber.

✓ **Subscribed**